In [7]:

```
import os
import logging
import pandas as pd
import numpy as np
from sqlalchemy import create_engine
from urllib.parse import quote_plus
import sys
import os

def ingest_db(df,table_name, engine, chunksize=50000):  # write 50k rows at a time
    df.to_sql(table_name, con=engine, if_exists='replace', index=False, chunksize=chunks

# ---------- Logging ----------
os.makedirs("log", exist_ok=True)
logging.basicConfig(
    filename="log/get_vendor_summary.log",
    filemode="a",
    level=logging.DEBUG,
    format="%(asctime)s - %(levelname)s - %(message)s",
    force=True,   # <-- This forces it to override any previous configs
)


# ---------- SQL ----------
QUERY = """
WITH pur AS (
    SELECT
        p.VendorNumber,
        p.VendorName,
        p.Brand,
        pr.Volume,
        p.PurchasePrice,
        pr.Price,
        SUM(Quantity) AS TotalQuantityPurchased,
        SUM(Dollars)  AS TotalAmountSpent
    FROM inventory.purchase_prices AS pr
    JOIN inventory.purchases      AS p
      ON pr.Brand = p.Brand
    WHERE p.PurchasePrice > 0
    GROUP BY p.VendorNumber, p.VendorName, p.Brand, p.PurchasePrice, pr.Price, pr.Volume
),
sls AS (
    SELECT
        VendorNo,
        VendorName,
        Brand,
        SalesPrice,
        SUM(ExciseTax)     AS TotalExcise,
        SUM(SalesQuantity) AS TotalQuantitySold,
        SUM(SalesDollars)  AS TotalEarned
    FROM inventory.sales
    GROUP BY VendorNo, VendorName, Brand, SalesPrice
),
frg AS (
    SELECT
        VendorNumber,
        VendorName,
```

```
        ROUND(SUM(Freight), 2) AS Total_Freight
    FROM inventory.vendor_invoice
    GROUP BY VendorNumber, VendorName
)
SELECT
    pur.VendorNumber                                    AS VendorNumber,
    pur.VendorName                                      AS VendorName,
    pur.Brand                                           AS Brand,
    pur.Price                                           AS ActualPricePerProduct,
    pur.PurchasePrice                                   AS PurPricePerProduct,
    pur.TotalQuantityPurchased                          AS TotalQuantityPurchased,
    pur.TotalAmountSpent                                AS TotalAmountSpent,
    pur.Volume                                          AS Volume,
    sls.SalesPrice                                      AS SalesPricePerQuantity,
    sls.TotalQuantitySold                               AS TotalQuantitySold,
    sls.TotalEarned                                     AS TotalEarned,
    sls.TotalExcise                                     AS TotalExcise,
    COALESCE(frg.Total_Freight, 0)                      AS TotalFreight
FROM pur
JOIN sls
  ON pur.VendorNumber = sls.VendorNo
 AND pur.Brand        = sls.Brand
LEFT JOIN frg
  ON pur.VendorNumber = frg.VendorNumber;
"""

# ---------- DB helpers ----------
def make_engine():
    # Avoid plain-text special chars breaking the URL
    password = quote_plus("PASSWORD@629")
    # Requires: pip install pymysql
    return create_engine(f"mysql+pymysql://root:{password}@localhost:3306/inventory?char

def create_vendor_summary(engine):
    logging.debug("Running vendor summary SQL query...")
    df = pd.read_sql_query(QUERY, engine)
    logging.debug("Vendor summary loaded: %d rows, %d cols", df.shape[0], df.shape[1])
    return df

def clean_data(df: pd.DataFrame) -> pd.DataFrame:
    logging.debug("Cleaning data...")
    df = df.copy()

    # Types
    numeric_cols = [
        "Volume", "TotalQuantitySold", "TotalQuantityPurchased",
        "TotalAmountSpent", "TotalEarned", "TotalExcise", "TotalFreight",
        "ActualPricePerProduct", "PurPricePerProduct", "SalesPricePerQuantity"
    ]
    for c in numeric_cols:
        if c in df.columns:
            df[c] = pd.to_numeric(df[c], errors="coerce")

    # Text cleanup
    if "VendorName" in df.columns:
        df["VendorName"] = df["VendorName"].astype(str).str.strip()

    # Metrics
    df["GrossProfit"] = df["TotalEarned"] - df["TotalAmountSpent"]
```

```
    # Handle divide-by-zero/nulls safely
    df["ProfitMargin"] = np.where(
        df["TotalAmountSpent"].fillna(0) != 0,
        (df["GrossProfit"] / df["TotalAmountSpent"]) * 100,
        np.nan
    )

    df["StockTurnOver"] = np.where(
        df["TotalQuantityPurchased"].fillna(0) != 0,
        df["TotalQuantitySold"] / df["TotalQuantityPurchased"],
        np.nan
    ).round(4)

    df["SalesToPurchaseRatio"] = np.where(
        df["TotalAmountSpent"].fillna(0) != 0,
        df["TotalEarned"] / df["TotalAmountSpent"],
        np.nan
    )

    logging.debug("Cleaning complete.")
    return df

# ---------- Main ----------
if __name__ == "__main__":
    try:
        logging.info("Creating SQLAlchemy engine...")
        engine = make_engine()

        logging.info("Creating Vendor Summary DataFrame...")
        summary_df = create_vendor_summary(engine)
        logging.info("Head:\n%s", summary_df.head().to_string())

        logging.info("Cleaning Data...")
        clean_df = clean_data(summary_df)
        logging.info("Head (clean):\n%s", clean_df.head().to_string())

        logging.info("Ingesting Data into MySQL...")
        # Make sure ingest_db(clean_df, table_name, engine) matches your notebook functi
        ingest_db(clean_df, "vendor_sales_summary", engine)
        logging.info("-------------- Ingestion completed ----------------")

    except Exception as e:
        logging.exception("Failed to build and ingest vendor_sales_summary: %s", e)
        raise
```

In [ ]: